AIPL Runtime Feature Matrix

7 implementations compared

Yasushi Kodama

2026-05-13

Overview

AIPL (Actor-based Intelligent Parallel Language, formerly AIPL) has grown into a **seven-runtime project**. This document provides a side-by-side comparison of every language and infrastructure feature across all implementations, grounded in source surveys.

Legend. ✓ = full support, △ = partial, × = not supported, – = not applicable.

The seven runtimes.

Short name	Source
Python (annotated)	`src/python-aipl/`
Python (inferred)	`src/python-aipl-inferred/`
OCaml	`src/*.ml`, `_build/.../repl_thread.exe`
JS-OCaml (server)	`src/app.js`, `console_server.js`, `ide.js` + OCaml `web_gateway`
JS-Browser (serverless)	`src/browser-abcl/`
JS-Node (server)	`src/node-aipl-server/` (Node HTTP server wrapping browser-abcl)
C (aipl2c)	`src/aipl2c.ml` codegen + `src/abcl_gui_runtime.c`

Column abbreviations in matrices below. **Py-A** = Python annotated, **Py-I** = Python inferred, **OCaml**, **JS-O** = JS via OCaml backend, **JS-B** = JS in browser, **JS-N** = JS in Node server, **C** = aipl2c codegen.

Inheritance relationships.

- **JS-OCaml** is a thin HTTP client over the OCaml backend, so its feature set inherits OCaml's (plus the new `/api/typecheck` endpoint).
- **JS-Node** wraps the same browser-abcl parser / type-checker / interpreter inside a Node.js HTTP server. Runtime feature set therefore mirrors JS-Browser.
- **Python (inferred)** imports parser, interpreter, AI, remote, and dashboard from `../python-aipl/`. All *runtime* features match Python (annotated); only the *static type-checker* layer differs (full Hindley-Milner instead of Phase 11+ annotation-driven).

Sample programs

Number of `.abcl` sample programs reachable by each runtime (core directory + AI integration + remote-actor demos):

	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
core samples	33	33	57	57	5	5	57
AI integration samples	11	11	8	8	0	0	8
remote-actor samples	8	8	1	1	0	0	1
total	**52**	**52**	**66**	**66**	**5**	**5**	**66**

Cross-cutting / not tied to a specific runtime:

- `aipl-self-host/`: 48 AIPL-in-AIPL self-host programs
- `docker/cross/samples/`: 4 cross-language interop samples

Note that the C runtime processes the same `.abcl` files as OCaml via `aipl2c`; of the 66 reachable samples, 48 currently pass the `aipl2c` smoke test (the remaining 9 in `abclc/` have pre-existing type ambiguities surfaced by our hard-fail policy).

What each sample exercises

Samples are grouped by the language or infrastructure feature they demonstrate. "Where" points to the source directory; "Target" lists the runtime(s) and codegen targets that actually execute the sample.

Target abbreviations: **Py** = python-aipl / python-aipl-inferred; **OCaml** = OCaml REPL (`abclc`); **C** = `aipl2c` → C+pthread; **SDL2** = `aipl2c` → C+SDL2 GUI binary; **Xinu** = `aipl2c --xinu` → Xinu-flavoured C; **Py-gen** = `aipl2c --python` → stand-alone Python; **Pony** = `aipl2c --pony` → Pony actor source; **Erlang** = `aipl2c --erlang` → Erlang module; **Go** = `aipl2c --go` → Go source; **Prolog** = `aipl2c --prolog` → SWI-Prolog `.pl`; **JS-B** = browser-abcl; **JS-N** = node-aipl-server.

Feature category	Sample(s)	Where	What it checks	Target
Basic actor model	`Hello`, `Counter`, `PingPong`	py-aipl + abclc	actor creation, `send`, field state, `self`-send	Py, OCaml, C, JS-B, JS-N
`now`/`future`/`await`	`NowFuture`, `CooperativeNowFuture`, `NowFutureDemo`	py-aipl + samples-ai + abclc/ai-samples	three message-passing forms in one program	Py, OCaml, C, JS-B, JS-N
Bounded mailboxes / select	`BoundedBuffer`, `bounded_buffer{,_visual}`	py-aipl + abclc + browser-abcl	producer/consumer, selective receive, visualisation	Py, OCaml, C, JS-B, JS-N
Dining philosophers	`Philosophers`, `philosophers{-1,-2}`, `Philosophers5{,_debug,_trace,Gui,Py,Xinu}`	py-aipl + abclc + browser-abcl	fork as actor, deadlock-free serialisation, SDL/Python/Xinu variants	Py, OCaml, C, SDL2, Py-gen, Xinu, JS-B, JS-N
`become` (class swap)	`become`, `bbecome`	abclc	runtime actor-class replacement	Py, OCaml
`select` / selective recv	`Channels`, `Channels2`	py-aipl/samples	typed channels, worker pool with request/result	Py
Method injection	`MethodPatch`	py-aipl/samples + abclc/	`add_method`/`remove_method` runtime patching	Py, OCaml

Feature category	Sample(s)	Where	What it checks	Target
Dynamic compile	`Dynamic`, `DynamicWorkerPool`	py-aipl/samples + abclc/	`compile()` builtin → runtime class generation	Py, OCaml
Top-level functions	`Functions`, `Signatures`	py-aipl/samples + abclc/	user functions, overload signatures, typeof	Py, OCaml
Records	`Records`	py-aipl/samples + abclc/	`{a:int, b:string}` literal, field access, structural typeof	Py, OCaml
Tuples	`Tuples`	py-aipl/samples + abclc/	positional, immutable, mixed slot types, nesting	Py, OCaml
Arrays	`Arrays`, `MultiDimArrays`	py-aipl/samples	static-sized, multi-dim, dynamic sizes	Py
Gradual type checker	`Typecheck`, `Typecheck11{b,c,de}`	py-aipl/samples	Phase 11 → 11e: literals, call-site validation, unions/generics, narrowing, length-tagged arrays	Py
Phase 11 typed counter	`Phase11_TypedCounter`	abclc	typed annotations / generics / typeof narrowing	OCaml
Phase 12 effects	`Effects`, `Phase12_EffectsLog`	py-aipl + abclc	capability-based `!{fs,ai,net,mut}` system	Py (static), OCaml (sample)
Phase 13 channels	`Phase13_Channels`	abclc	CSP channels (design doc + runtime in Py)	Py (runtime), OCaml (sample)
Phase 14 linear types	`Linear`, `Linear2`, `Phase14_Linear`	py-aipl + abclc	use-after-move; DB transaction with linear handle	Py (static), OCaml (sample)
Phase 15 owned fields	`Owned`, `Owned_violations`, `Phase15_Owned`	py-aipl + abclc	`pub` field visibility, intentional violations	Py (static), OCaml (sample)
Phase 16 transient cast	`Transient`, `Transient_violation`	py-aipl/samples	runtime type cast at any-boundary	Py
Phase 17 structured conc.	`Phase17_StructuredConc`	py-aipl/samples	`scope { future ... }` auto-joining	Py
Session types / protocol traces	`SessionTyped`	py-aipl/samples-ai	runtime-checked typed session protocols (arg + reply types); order-only protocol traces; AIOS service registry	Py
AI integration (mock + real)	`AIActor`, `AIChain`, `AIChainReal`, `AIHello`, `MultiProvider`	py-aipl + samples-ai + abclc/ai-samples	LLM-backed actors; multi-provider; chained pipeline	Py, OCaml
AI governance	`Budgeted`	py-aipl/samples-ai + abclc/ai-samples	token budget, concurrency cap, fallback chain	Py, OCaml
AI cooperative pattern	`CooperativeNowFuture{,-jp,-jp-remote}`, `CooperativeSolve{,-jp,Remote,Remote-jp}`, `Reviewer`, `Fanout`, `PriorityFanout`	py-aipl/samples-ai, abclc/ai-samples	Planner→Solver→Reviewer; fan-out aggregator; priority routing	Py, OCaml

Feature category	Sample(s)	Where	What it checks	Target
Remote actors	`client`, `server`, `coordinator`, `solver`, `verifier`, `reviewer_node*`, `RemoteCalcClient/Server`	py-aipl/samples-remote + abclc/samples-remote + abclc/ai-samples	HTTP cross-machine sends; HMAC-signed coordination	Py, OCaml
Web / dashboard	`web_calc`, `SiteGen`	abclc + py-aipl/samples	embedded HTTP gateway; static-site generator	Py (SiteGen), OCaml (web_calc)
GUI / SDL2	`Rotate{One,Three,Four}Lines{,Gui}`, `MultiLineSpin`, `Philosophers5Gui`, `BoundedBufferGui`, `DisasterReturnGui`, `LineDrawer`, `window`	abclc	SDL2-backed GUI codegen via aipl2c	SDL2 (via aipl2c)
Python codegen target	`BoundedBufferPy`, `Philosophers5Py`, `Rotate4LinesPy`	abclc	`aipl2c --python` emits stand-alone Python	Py-gen
Xinu (embedded OS) target	`BoundedBufferXinu`, `Philosophers5Xinu`, `Rotate4LinesXinu`	abclc	`aipl2c --xinu` emits Xinu-flavoured C	Xinu
Pony codegen target	`Hello`, `counter` (verified); `PingPong` (xfail)	abclc	`aipl2c --pony` emits Pony source; `class` → `actor`, methods → `be`; two-step `_aipl_init` decouples construction from init body	Pony
Erlang codegen target	`Hello`, `counter` (verified); `PingPong` (xfail)	abclc	`aipl2c --erlang` emits an `.erl` module; `class` → `spawn` + `receive` loop; fields → loop args with versioned variables on assign	Erlang
Go codegen target	`Hello`, `counter` (verified); `PingPong` (xfail)	abclc	`aipl2c --go` emits a single `main.go`; `class` → struct + goroutine; methods → typed message structs + helper methods + type-switch dispatch	Go
Prolog codegen target	`Hello`, `counter` (verified); `PingPong` (xfail)	abclc	`aipl2c --prolog` emits a single SWI-Prolog `.pl` using `library(thread)`; `class` → `c_loop(Fields)` thread + `thread_get_message` + `Msg = m(Args) -> body` dispatch; expressions hoisted to goals (`X is A + B`, `format(atom(S), ...)`)	Prolog

Feature category	Sample(s)	Where	What it checks	Target
LLVM codegen target	`Hello`, `counter` (verified)	abclc	`aipl2c --llvm` emits the default C with clang-specific `__attribute__((hot))` / `((cold))` on the actor / watchdog / print paths, plus a header banner listing `clang -O2 -pthread` (native), `clang -emit-llvm -S` (text IR), and `clang -emit-llvm -c + lli` (interpret) workflows	LLVM
OpenMP codegen target	`Hello`, `counter` (verified)	abclc	`aipl2c --openmp` emits C + `<omp.h>`; the initial actor-spawn loop is converted to `#pragma omp parallel for schedule(dynamic)` and the message counter uses `#pragma omp atomic`. Build with `gcc-15 -fopenmp` or `clang -fopenmp`. pthreads still drive per-actor message loops (coexist with OpenMP)	OpenMP
Drone / simulation	`drone_simulator`	browser-abcl	obstacle-aware drone swarm with comm + view range	JS-B, JS-N
Trace / minimal repros	`H`, `P`, `T*`, `LD*`, `MS`, `AA`, `PP`, `PH`, `line*`, `Philosophers5_{debug,trace}`	abclc	reduced repro cases used during runtime / TLA+ / Spin model-checking	OCaml

Core language

Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
method injection (`add_method`/`remove_method`)	✓	✓	✓	✓	×	×	×
`now` synchronous send	✓	✓	✓	✓	✓	✓	✓
`future` async send	✓	✓	✓	✓	✓	✓	✓
`await` future block	✓	✓	✓	✓	✓	✓	✓
`send` fire-and-forget	✓	✓	✓	✓	✓	✓	✓
`become` actor class swap	✓	✓	✓	✓	×	×	×
`select` selective receive	✓	✓	✓	✓	✓	✓	×
top-level functions	✓	✓	✓	✓	×	×	✓
signatures / overloads	×	×	✓	✓	×	×	×
dynamic compile (`compile()`)	✓	✓	✓	✓	×	×	×
worker pool / `DynamicWorkerPool`	✓	✓	✓	✓	△	△	×
text file I/O (`read_file`/`write_file`/`append_file`/`file_exists`)	✓	✓	✓	✓	✓[a]	✓[a]	×
image I/O (`image_create`/`image_load`/`image_save`/`image_size`/`image_pixel`/`image_set_pixel`)	✓[b]	✓[b]	✓[c]	✓[c]	✓[a,c]	✓[a,c]	×

[a] JS-B / JS-N の I/O ビルトインは host が Node の `fs` を注入したとき (`server.mjs` が自動で行う) のみ動作。ブラウザ内では"not available" を投げる。
[b] Py-A / Py-I は Pillow 経由で PNG / JPEG / GIF / WebP を判定して読み書き。
[c] OCaml / JS-O / JS-B / JS-N は純ランタイムの PPM (P6) バックエンド (RGBA メモリ表現、保存時はアルファ破棄)。PNG は外部ライブラリが必要。

Type system

Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
type inference	△trace	✓HM	✓HM	✓HM	✓flow	✓flow	✓HM+spec
type annotations	✓	△ignored	✓	✓	×	×	×
records `{a:int, b:str}`	✓	✓	✓	✓	×	×	△type only
tuples `(1, "a")`	✓	✓	✓	✓	×	×	×
arrays (typed, multi-dim)	✓	✓	✓	✓	✓	✓	△
generics on functions	×	✓Forall	✓Forall	✓	×	×	×

Phase 11+ advanced features

Python (inferred) runs HM only, so the annotation-driven Phase 11+ static checks that Python (annotated) performs are not re-implemented. Programs still execute at runtime via the shared interpreter, but the static guarantees from Phase 12 / 14 / 15 / 16 are lost. Phase 13 (channels), Phase 17 (`scope`), and the session-type / protocol-trace / AIOS-registry families

are runtime features, so they remain available in Py-I.

Session types here are *runtime-checked* and explicitly separate from HM inference: they observe values flowing over the wire at runtime and validate them against a declared protocol spec (`"actor.method(arg_t1, ...) ! ret_type -> ..."`). Violations are recorded into `session_events()` and counted; the program does not abort.

Feature	**Py-A**	**Py-I**	**OCaml**	**JS-O**	**JS-B**	**JS-N**	**C**
channels (CSP-style) —Phase 13	✓	✓	×	×	×	×	×
linear types / use-after-move —Phase 14	✓	×	×	×	×	×	×
owned / pub fields —Phase 15	✓	×	×	×	×	×	×
effects `!{fs,ai,net,mut}` —Phase 12	✓	×	×	×	×	×	×
structured concurrency `scope` —Phase 17	✓	✓	×	×	×	×	×
transient cast at any-boundary —Phase 16	✓	×	×	×	×	×	×
session types (runtime-checked typed protocols)	✓	✓	×	×	×	×	×
protocol traces (order-only)	✓	✓	×	×	×	×	×
AIOS service registry (alias-resolved send)	✓	✓	×	×	×	×	×

Phase C–E2 type inference (Python annotated only, added 2026-05-17/18)

Phase 11–17 の checker layer に加え、Python (annotated) ランタイムは **constraint-based Hindley–Milner + Z3 refinement** 推論系を別モジュール `aipl_inference.py` (~1255 LOC) として持つ。Py-I (`python-aipl-inferred/`) は依然 trace-based HM のままで, これらの新 phase は取り込まれていない。

ID	Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
CE-1	Phase C: constraint-based HM + Z3 refinement (Int)	✓	×	△**	△**	×	×	×
CE-2	Phase D-1: cross-class inference	✓	×	✓#	✓#	×	×	×
CE-3	Phase E-α: `where` 句 in AIPL grammar	✓	×	△*	△*	×	×	×
CE-4	Phase E-β: actor field 共有 (method 横断)	✓	×	✓#	✓#	×	×	×
CE-5	Phase E-γ: record structural typing	✓	×	×	×	×	×	×
CE-6	Phase E-γ-R: Real / Rat refinement (Z3 Real)	✓	×	△**	△**	×	×	×
CE-7	Phase E-2: typeck × inference 統合 CLI `--check`	✓	×	×	×	×	×	×
CE-8	`--infer` standalone CLI (Phase D-4)	✓	×	×	×	×	×	×
CE-9	refinement vacuously-false detection (declaration-time, Z3 unsat)	✓	×	△**	△**	×	×	×

サンプル: `aice-pi-evolution/experiments/2026-05-17_aipl_v2_type_inference/samples/feature_{a..g}/` (7 feature × 3 = 21 demo + 27/27 unit tests).

* CE-3: OCaml は Phase O-2.a (2026-05-18) で parsing 達成.`lexer.mll` に `where/and/or/not` キーワード, `parser.mly` に `type_expr WHERE refine_or` 規則, `ast.ml` に `TyERefined of type_expr * refine_pred` を追加. サンプル: `abclc/WhereClause.abcl`.

** CE-1 / CE-6 / CE-9: OCaml は Phase O-2.b (2026-05-18) で SMT-LIB 2 + z3 CLI バックエンドを実装, Phase O-2.c (同日) で Real 理論にも対応. `src/refinement.ml` が `Ast.refine_pred` を `smt_sort` (Int / Real) パラメータ付きで SMT-LIB 2 にレンダし `z3 -in -t:5000` に流す.`AIPL_REFINE_CHECK=1` で typecheck pass が vacuously-false な refinement に `[refine warning]` を吐く. z3 不在時は `Deferred` で graceful fallback. サンプル: `abclc/WhereVacuous.abcl` (Int), `abclc/WhereVacuousReal.abcl` (Real).

CE-2 / CE-4: OCaml は Phase O-2.d (2026-05-18) で cross-class 推論を強化.`preinfer_all_classes` が declared return-type を尊重し,`Now` / `Future` expression が `class_method_schemes` 越しに actual ret 型を返すよう変更. `TypedVarDecl` の unify 失敗を Type_error 化. actor field の cross-method 一貫性 (Int/String mix 検出) も自動取得.

OCaml ランタイムも HM 推論を持つが (#12 で ✓), その他の refinement 機能 (CE-1, CE-6, CE-9) と record structural typing (CE-5) は今後の課題.

AIPL v2 Distributed runtime (`aipl_dist`, added 2026-05-18)

進化計算 (MAP-Elites 8 seed × 30 gen × 2 run) で発見された分散ランタイム設計 (I0003 balanced / I0023 hang-resilience / I0036 Erlang OTP の 3 候補) を `aipl_dist.py` (591 LOC) として実装。AIPL 言語仕様は不変, 既存サンプルは無改変で同じ挙動。**`AIPL_DIST_ENABLE=1` で全機能 opt-in**。

ID	Feature (env var)	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
DR-1	I-1 `env_var_routing` (`AIPL_ROUTE`)	✓	×	✓	✓	×	×	×
DR-2	I-2 `structured_log` (`AIPL_DIST_LOG_FILE`)	✓	×	✓	✓	×	×	×
DR-3	I-3 `token_budget_aware` (`AIPL_DIST_RPM` / `TPM`)	✓	×	✓	✓	×	×	×
DR-4	I-4 `checkpoint_and_resume` (`AIPL_DIST_CHECKPOINT_DIR`)	✓	×	✓	✓	×	×	×
DR-5	IQ `quarantine_and_skip` (`AIPL_DIST_QUARANTINE_TTL`)	✓	×	✓	✓	×	×	×
DR-6	IM-1 `quorum_replicate` (`AIPL_DIST_QUORUM_PROVIDERS`)	✓	×	✓	✓	×	×	×
DR-7	IM-2 `subtree_quarantine` (`AIPL_DIST_SUBTREE_QUARANTINE`)	✓	×	✓	✓	×	×	×
DR-8	spawn-tree tracking (TLS-auto parent)	✓	×	✓	✓	×	×	×
DR-9	runtime hooks (interp / actor / call_ai) — opt-in	✓	×	✓	✓	×	×	×

実装位置: `src/python-aipl/aipl_dist.py` (591 LOC, 新規) + `aipl_interp.py` (+27 行 spawn hook) + `aipl_runtime.py` (+13 行 error hook) + `aipl_ai.py` (+13 行 budget hook).
サンプル: `aice-pi-evolution/experiments/2026-05-17_aipl_v2_type_inference/IMPL_I0003_MVP/samples/` (8 feature × 3 = 24 demo + 27/27 unit tests).
設計探索の出自: 同 dir の `AIPL_v2_Distributed.{aice,ga.json,schema.json}` を MAP-Elites で 24 min × 2 run。詳細は `IMPL_I0003_MVP/IMPL_DESIGN.md` ~ `IMPL_I0036_REPORT.md` 参照。

OCaml は Phase O-1 + O-1.5 で全 9 機能 ✓達成.DR-3/DR-6 は `Ai.call_gemini` へ自動 wire 済,DR-8 は per-thread `current_actor` TLS で parent を自動取得.

Networking, AI, infrastructure

Feature	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
remote actors (HTTP)	✓	✓	✓[d]	✓[d]	×	△srv only	✓via OCaml
WebSocket	✓`ws_listen`/`send`/`close` builtins (web-sockets lib)	✓(=Py-A)	✓built-in (verified 101)	✓via OCaml	✓browser-native	✓`/ws?sid=` + `/api/broadcast` (ws lib)	✓`abcl_ws_runtime.c` + lib-uv sock-ets
AI integration (`ai_call`)	✓stream/img	✓	✓	✓	✓mock	✓mock	✓
provider arg (`ai_call([1|2|3,] prompt)`) —1=gemini, 2=anthropic, 3=openai	✓	✓	✓	✓	✓	✓	✓(via OCaml)
`now actor.m(...)` + `ai_call` inside method (blocking)	✓	✓	✓	✓	✓	✓	×
`future actor.m(...)` + `await(f)` + `ai_call`	✓	✓	✓	✓	✓	✓	×
`send` + callback pattern with `ai_call`	✓	✓	✓	✓	✓	✓	×
AI governance (budget/concurrent/fallback)	✓	✓	✓	✓	×	×	✓
HMAC-signed remote send	✓	✓	✓[e]	✓[e]	×	×	✓
persistent actor state	✓	✓	×	×	×	×	×
live dashboard (SSE)	✓	✓	△polling	△	×	×	✓
`/api/typecheck` JSON endpoint	×	×	✓	✓	×	✓	–
`/api/run` JSON endpoint	×	×	×	×	×	✓	–

Actor concurrency model

Every AIPL implementation runs each actor as a separate concurrent unit, but the kind of unit differs by 3–4 orders of magnitude in weight. This affects how many actors a program can usefully spawn, how messages interleave, and whether blocking I/O in one actor stalls the whole runtime.

Runtime / codegen target	Model	Per-actor unit	Source pointer
Python (annotated)	**1:1 OS thread**	`threading.Thread(daemon=True)`	`aipl_runtime.py:92`
Python (inferred)	**1:1 OS thread**	(inherits from python-aipl)	(re-uses interp)
OCaml	**1:1 OS thread**	`Thread.create`	`eval_thread.ml:~1168`
JS-OCaml (server)	**1:1 OS thread**	(= OCaml backend threads)	(= OCaml)
JS-Browser (browser-abcl)	**cooperative event loop**	`setTimeout`-driven mailbox drain	`runtime.js:~233`
JS-Node (node-aipl-server)	**cooperative event loop**	(= browser-abcl runtime)	re-export
C (default)	**1:1 OS thread**	`pthread_create`	`c_translator.ml:~721,790`
C + SDL2	**1:1 OS thread**	`pthread_create`	(same as default C)
C + Xinu	**1:1 OS process** (kernel-level)	Xinu `create()` + `ready()`	`c_translator.ml:~1077`

Runtime / codegen target	Model	Per-actor unit	Source pointer
C → Python codegen	**1:1 OS thread**	`threading.Thread` (`o._spawn()`)	`c_translator.ml:~1847`
C → Pony codegen	**M:N lightweight** (runtime-scheduled)	Pony actor (work-stealing)	implicit in generated `actor`
C → Erlang codegen	**M:N lightweight** (BEAM)	BEAM process (`spawn(fun()->...end)`)	`c_translator.ml:~2413`
C → Go codegen	**M:N lightweight** (Go runtime)	goroutine (`go c.run()`)	`c_translator.ml:~2728`
C → Prolog codegen	**1:1 OS thread**	SWI thread (`thread_create/3`)	`c_translator.ml:~3060`

Three concurrency tiers

- **1:1 OS thread / process** (Python, OCaml, C, C-SDL2, Xinu, C→Python, C→Prolog): each actor is a kernel-scheduled thread. Simple mental model; blocking I/O in one actor doesn't stall others. Scales to ∼100–1000 actors before kernel overhead dominates.
- **M:N lightweight** (Pony, Erlang, Go): actors are scheduled by the language runtime onto a small pool of OS threads. Scales to $\sim 10^5$–10^7 actors. Blocking syscalls in one actor may stall its scheduler unless the runtime intercepts them (Erlang's BEAM intercepts; Go's runtime intercepts via `netpoll`; Pony's runtime requires non-blocking idioms).
- **Cooperative event loop** (browser-abcl, node-aipl-server): single OS thread; mailboxes are drained turn-by-turn through `setTimeout(0)` re-entry into the event loop. No real parallelism, but message-level interleaving is preserved. A long synchronous computation in one actor *will* stall all others — by design, since this is the only concurrency model the browser DOM allows.

Cross-cutting implication

The AIPL programming model (`send` / `now` / `future` / `await`) is preserved across all three tiers — the choice of target picks a point on the cost/scale curve without changing the program text. The same `.abcl` file:

- runs ∼1000 actors fine on Python / OCaml / C (OS-thread tier);
- scales to millions of actors on Erlang / Go / Pony codegen;
- runs in a browser sandbox via JS-Browser at the cost of single-threaded execution.

C-version-specific codegen targets

Target	C
C + pthread standalone binary	✓
C + SDL2 GUI binary (1178-line runtime)	✓
Xinu embedded-OS target (`--xinu`)	✓
Python target (`--python`)	✓
Pony target (`--pony`) —`class`→`actor`, methods→`be`, two-step `_aipl_init`	✓
Erlang target (`--erlang`) —`class`→`spawn`+`receive` loop, fields→versioned loop args	✓
Go target (`--go`) —`class`→struct+goroutine, methods→typed message structs + type-switch dispatch	✓
Prolog target (`--prolog`) —`class`→SWI thread + receive loop, expressions hoisted to goals (`is/2`, `format(atom(...))`)	✓

Key observations

Python (annotated) is the most feature-complete runtime

Of the 30 surveyed features, Python (annotated) ticks roughly 28. It owns the full Phase 11–17 stack (channels, linear types, owned fields, effects, structured concurrency, transient casts), method injection, dynamic compile, persistent actor state, full AI integration (streaming + multimodal images), and Pillow-backed image I/O (PNG / JPEG / GIF / WebP).

Python (inferred) — Python's HM-typed twin

Runtime features match Python (annotated) exactly because the interpreter is shared. The trade is on the static side: the Phase 11+ annotation-driven checks are replaced with a Hindley-Milner inference pass. Inferred types cross-verified against OCaml: on shared samples (`Hello.abcl`, `counter.abcl`) the two runtimes produce identical type schemes.

OCaml — feature parity with Python on the core

After the OCaml feature catch-up, the runtime supports **method injection** (`add_method`/`remove_method`), **dynamic compile** (`compile()` + `spawn()`), **top-level functions** with **generics** (`function id(x: T) -> T`), **type annotations**, **records & tuples**, **sized arrays** (`var x[N][M]`), and **text/image file I/O** (PPM backend for images). Phase 11+ effect / linear / owned / transient features still only exist as `.abcl` design-document samples — not statically enforced. Solid `become` / `select` / WebSocket (built into `web_gateway.ml`) and HTTP remote actor calls (`send` / `now` / `future`, error-tolerant) with **HMAC-signed traffic** (`ABCL_REMOTE_SECRET` 環境変数, 純 OCaml HMAC-SHA256 in `src/hmac_sha256.ml`, Python / C 版とワイヤ互換).

[d] OCaml の remote client (`src/remote_client.ml`) は `send` (fire-and-forget) / `now` (同期 reply) / `future` (並列 + await) の 3 形式すべてをサポート。ECONNREFUSED / timeout は stderr に記録され、呼び出し側のアクターは生存する。エンドツーエンド検証は `src/test_remote_actor.sh` (9/9)。JS-O は OCaml バックエンド経由でこれを継承する。

[e] 純 OCaml HMAC-SHA256 (`src/hmac_sha256.ml`, RFC 6234 / 4231 テストベクタで verify) を sender と receiver の両側で使う。`ABCL_REMOTE_SECRET` が両端でセットされたとき、全リクエストに `X-ABCL-Sig: <hex>` が付与される。`web_gateway.ml` の `verify_hmac_or_reject` は不一致なら 401 を返す。`python-aipl/aipl_remote.py` と C ランタイムにワイヤ互換。以前は受信側が `openssl dgst` をサブプロセス起動していたが、インプロセスの純 OCaml 版に置換 (リクエスト毎の fork 排除)。

JS-OCaml ≈ OCaml

A thin HTTP client (`src/app.js` etc.) over the OCaml backend exposed by `web_gateway.ml`. Every OCaml-side feature — method injection, dynamic compile, top-level functions, generics, type annotations, file & image I/O — is reachable via `/api/repl` from a browser or `curl`. The OCaml-specific differentiator is the `/api/typecheck` JSON endpoint.

JS-Browser — minimal in-browser engine

Basic actor model plus flow-sensitive type inference. Recently picked up **sized arrays** (`var x[N]`) and **text & image file I/O** (host-injected `fs`; throws in browsers). No `become`, no channels, no remote, no method injection. AI integration exists but is mock-only.

JS-Node ≈ JS-Browser + fs

Wraps the browser-abcl runtime inside a Node.js HTTP server (`src/node-aipl-server/server.mjs`); auto-injects Node's `fs` so the text & image I/O builtins are functional. Exposes `/api/typecheck` and `/api/run` for headless use. Same flow-sensitive type inference as browser-abcl.

C codegen — broadest target diversity

`become` and `select` remain codegen-unsupported (emit `/* unsupported */`), but the codegen inherits OCaml's HMAC, SSE, and AI governance, and offers **9 backends** from one front-end:

- C + pthread (default)
- **C + LLVM / clang** (`--llvm`: adds `__attribute__((hot/cold))` + headers for `clang -emit-llvm`)
- **C + OpenMP** (`--openmp`: `#pragma omp parallel for` spawn loop + `#pragma omp atomic` counter)
- C + SDL2 (GUI demos)

- Xinu / Python / Pony / Erlang / Go / SWI-Prolog

Fields, parameters, and locals are specialised to native C types via HM inference + per-class field-type registry.

Method injection — now in three runtime families

Both Python variants and the OCaml runtime (with JS-OCaml piggy-backing on it) implement mutable method dispatch tables with runtime `add_method` / `remove_method` / `methods_of`. `browser-abcl` and `node-aipl-server` still lack even `become` and rely on the unchanging method table baked at parse time.

Type inference cross-verification

For shared samples (`abclc/Hello.abcl`, `abclc/counter.abcl`), the three HM-based runtimes (**OCaml**, **Python-inferred**, **C**) produce identical inferred types:

```
Hello:   count : float
         init  : (int) -> unit
         greet : () -> unit
         inc   : () -> unit

Counter: count : float
         inc   : () -> unit
         dec   : (float) -> unit    <- monomorphized via call site
```

The flow-sensitive variants (**JS-Browser**, **JS-Node**) reach the same answers on shared samples within the limits of their algorithm —field types match, but method signature schemes are not produced the same way.

Phase 11+ implementation gap

The full Phase 11–17 stack is implemented only in Python (annotated). This represents AIPL's language-evolution frontier; Python (annotated) serves as the research runtime. Python (inferred) keeps the runtime side but trades the static checks for HM-style polymorphic inference — a different research axis (declarative typing vs. annotated capabilities).

Post-Phase-11 (added 2026-05-17/18)

領域	Py-A 達成	他ランタイム	関連 ID
Phase C–E2 型推論 (HM + Z3 refinement)	9/9	0/9	CE-1 ∼ CE-9
AIPL v2 Distributed (`aipl_dist`)	9/9	**OCaml 9/9** (= JS-O)	DR-1 ∼ DR-9

Py-A 単独で進化計算由来の 18 機能が opt-in 利用可能 (`AIPL_DIST_ENABLE=1` または `--check` / `--infer` で発火, デフォルトは従来挙動と同一)。OCaml 側への移植 path:

- 型推論側: `--infer` の HM-with-refinement を OCaml HM (`src/infer.ml`) に統合すれば CE-1, CE-2, CE-4 は取り込み可能. `where` 句 (CE-3) は OCaml lexer/parser 修正が必要.Z3 backend (CE-1, CE-6, CE-9) は別プロセス via JSON が現実的.
- ランタイム側: DR-1 / DR-2 / DR-4 / DR-5 / DR-8 / DR-9 は OCaml `eval_thread.ml` + `aipl_remote.ml` への hook 追加 (~200 LOC) で実装可能.DR-3 / DR-6 は AI client 層, DR-7 は spawn-tree registry が要る.

Generated 2026-05-14, updated 2026-05-18. Cumulative through: OCaml feature catch-up (records / tuples / dynamic compile / method injection / top-level functions / generics / type annotations / sized arrays / text & image I/O); C-codegen LLVM / OpenMP targets; `python-aipl-inferred` (commit `270f291`) and `node-aipl-server` (commit `adeb0b6`); **Phase C–E2 constraint-based HM + Z3 refinement** (`aipl_inference.py`, 1255 LOC; sections CE-1..CE-9); **AIPL v2 Distributed runtime** (`aipl_dist.py`, 591 LOC; sections DR-1..DR-9), discovered by MAP-Elites GA + MVP-implemented for I0003 / I0023 / I0036 winners.

For source pointers, run `grep` against the files listed in each runtime's source column.